Introduction to Programming:

LIBRARY FUNCTIONS: INPUT, OUTPUT, MATHEMATICS, DATE AND TIME

Prepared By:

Darsha Chauhan

Outline

- Reading Character from Keyboard
- Printing Character on Screen
- Reading String from Keyboard
- Printing String on Screen
- Formatting input and Output
- Time Functions: difftime, clock, time
- Math Functions: abs, fmod, log, log2, pow, sqrt, ceil, floor.

Input & output functions

- getchar() :

This function reads a single character from terminal.

- putchar():

This function displays the character passed to it on the screen.

- gets() :

This function reads a line from stdin into the buffer pointed to by s until either a terminating newline or EOF (End of File) occurs.

- puts():

This function writes the string 's' and 'a' trailing newline to stdout.



getchar() & putchar()

```
#include <stdio.h>

void main( )
{
        int c;

        printf("Enter a character: ");

        /* Take a character as input */

        c = getchar();

        /* Display the character */

        printf("Entered character is: ");

        putchar(c);

}
```

o/p:
Enter a character: a
Entered character is: a

gets() & puts()

```
#include <stdio.h>
void main( )
{
        /*Character array of length 100*/

        char str[100];

        printf("Enter a string: ");

        /* Take a string as input */

        gets( str );

        /* Display the string */

        printf("Entered string is: ");

        puts( str );
}
```

O/p:
Enter a string: india
Entered string is: india

Formatted Input & output

- The general form of printf() is as follows:

  printf("Control string", arg1, arg2, ..., argn);

- Control string consists of three types of items:
  1. Characters that will be printed on the screen as they appear
  2. Format specifications that define the output format for display of each item
  3. Escape sequence characters such as \n, \t and \b
- The simple format specifications has following form:

  %w.p type-specifier

  w = integer no that specifies total no of columns for o/p

  p=integer no that specifies the no of digits to the right decimal point / characters to be printed from string

- Both w & p are optional

Formatted Input & output

printf("%d",, 9876);



printf("%6d",, 9876);



printf("%2d",, 9876);



printf("%-6d",, 9876);



printf("%06d",, 9876);



Formatted Input & output

float f= 98.7654;

printf("%7.4f",y);



printf("%7.2f",y);



printf("%-7.2f",y);



printf("%f",y);



printf("%10.2e",y);



printf("%-10.2e",y);



Formatted Input & output

float f= 98.7654;

printf("%7.4f",y);



printf("%7.2f",y);



printf("%-7.2f",y);



printf("%f",y);



printf("%10.2e",y);



printf("%-10.2e",y);



Formatted Input & output

str= “New Delhi”;

printf(“%s”,str);



printf(“%12s”,str);



printf(“%12.4s”,str);



printf(“%-12s”,str);



printf(“%5s”,str);



Math.h functions

Function	Data type returned		Example
abs()	double	Return absolute value of d	abs(-5)= 5
fmod(d1, d2)	double	Return the reminder of d1 /d2 (same sign as d1)	fmod(-10, 21)=-1
log(d)	Double	Return natural logarithm of d	log(2)=0.673
log2(d)	Double	Return logarithm of d (base 2)	log2(2) = 1
pow(d1,d2)	Double	Return d1 raised to th d2 power	pow(2,3) =8
sqrt(d)	Double	Return square root of d	sqrt(16) = 4
ceil(d)	Double	Return value rounded upto the next higher integer	ceil(3.1) = 3
floor(d)	Double	Return value rounded down to the next lower integer	floor(3.9) = 3

time.h functions

- The **time.h** header file contains definitions of functions to get and manipulate date and time information.

- It describes three time-related data types.
  - **clock_t**: clock_t represents the date as an integer which is a part of the calendar time.

  - **time_t**: time_t represents the clock time as an integer which is a part of the calendar time.

  - **struct tm**: struct tm holds the date and time
  - It also contains CLOCKS_PER_SEC macro which holds the number of time does the system clock ticks per second.



time.h functions

```
struct tm {
        int tm_sec; // seconds,  range 0 to 59
        int tm_min; // minutes, range 0 to 59
        int tm_hour; // hours, range 0 to 23
        int tm_mday; // day of the month, range 1 to 31
        int tm_mon; // month, range 0 to 11
        int tm_year; // The number of years since 1900
        int tm_wday; // day of the week, range 0 to 6
        int tm_yday; // day in the year, range 0 to 365
        int tm_isdst; // daylight saving time
}
```



time.h functions: clock()

- The C library function **clock_t clock(void)** returns the number of clock ticks elapsed since the program was launched. To get the number of seconds used by the CPU, you will need to divide by CLOCKS_PER_SEC.
- On a 32 bit system where CLOCKS_PER_SEC equals 1000000 this function will return the same value approximately every 72 minutes.

time.h functions

Function	
clock()	• This function returns the processor time consumed by a program • It may be an integer or a floating-point type. Its values are counts of *clock ticks* since some arbitrary event in the past. The number of clock ticks per second is system-specific.
difftime(time_t *time2*, time_t *time1*)	• This function returns the difference between the times provided.(in seconds)
time()	• This function returns the calendar-time equivalent using data-type time_t. • time_t is an integer type and its values represent the number of seconds elapsed since the *epoch*, which is 00:00:00 on January 1, 1970, Coordinated Universal Time.



#Thank You

From @:Darsha

☺☺☺